WebMARS

Enhancement Plan & Two-Week Sprint

Snippet Sharing, User Accounts, Run History,
Tools Polish, and UX Bug Fixes

Web-based MIPS Assembly Runtime Simulator

webmarsimulator.com

Field	Value
Document version	1.0
Date	May 14, 2026
Project version target	WebMARS v1.2 (frontend) + webmars-api v1.2
Sprint length	10 working days (2 calendar weeks)
Team	Bryan Djenabia, Landon Clay, Zachary Gass

Field	Value
Frontend repo	Webmarssimulator/WebMARS
Backend repo	landclay715/webmars-api
Status	Plan approved, ready to execute

Table of Contents

1. Executive Summary

WebMARS today is a fully functional in-browser MIPS simulator at webmarsimulator.com. The next two weeks turn it into a small social product: users will be able to register, save their MIPS programs to an account, share them with a public link, and see a history of every program they have run.

Three parallel tracks of work, one per engineer, run for ten working days. The plan is sized so each engineer ships one to three commits per day. Days 1 through 5 prove out the foundations (security fixes, persistence, theme bug, snippet-account linking). Days 6 through 10 add the new features on top (run history, public/private, polish, deployment).

1.1 What the team is shipping

- Backend v1.2 expanded: snippets owned by users, public/private visibility flag, run-history table, most-run aggregation endpoint, hardened authentication, proper error responses, CORS configured for the deployed frontend.
- Frontend v1.2 integrated: login and register modals, share link with copy-to-clipboard, my-snippets drawer, public-snippet feed, run-history panel, auto-save to localStorage.
- Critical UX bugs fixed: code persistence across page refresh, Monaco editor honoring light vs dark theme, error toasts for failed network calls, loading states for async operations.
- Tool enhancements: rem pseudo-instruction added to the assembler, bitmap display gets 8x8 and 16x16 grid sizes, screen magnifier rebuilt as a working DOM-clone implementation (no longer a placeholder).
- Deployed: backend live on Render, frontend redeployed to Vercel with environment-aware API base URL.

1.2 Why this order

Security comes first because the JWT signing key and Postgres password are both visible in the git history today. A single git clone exposes the keys. No new feature work begins until those are rotated and moved to environment variables.

Persistence and the light-mode bug come next because they are the loudest user complaints and the fixes are small. They build morale and clear the deck for the bigger feature work that follows.

The account-linking schema work happens before any new endpoints, because the snippet table needs an owner_id column before public/private visibility makes sense.

Tool work (rem, bitmap, magnifier) runs in the background on Zachary's track because it is fully independent of everything else and gives him steady wins while Landon and Bryan converge on the larger account features.

2. Critical Bugs and UX Issues

Each issue below was confirmed against the actual code in the two repositories. File paths and line numbers are real and current. None of these are speculative.

2.1 Code disappears when you refresh the page

[FRONTEND] **[HIGH]** Severity is high because students lose work and there is no recovery path.

Where the bug lives

File: src/hooks/useSimulator.ts

The Zustand store persists two things to localStorage: number-base preference (line 145, 158) and panel layout (line 166, 190). The source code in the editor is not persisted. When the user reloads the page, the source field resets to the default Hello-MIPS example and any unsaved work is gone.

Why it happens

There is no persist middleware wrapping the store, and no manual write to localStorage for the source slice. The store is built with plain create(set, get) instead of create(persist((set, get) => ...)). Refreshing the tab discards the in-memory store and the editor mounts with the default source.

How to fix it

Add Zustand's persist middleware around the store, scoped to a small set of safe keys (source, snippet metadata, theme preference). Do not persist the entire store because the simulator state (registers, memory, breakpoints) should reset on reload.

```
// src/hooks/useSimulator.ts
import { persist, createJSONStorage } from 'zustand/middleware'

export const useSimulator = create<SimulatorStore>()(
  persist(
    (set, get) => ({
      // ...existing store body unchanged...
    }),
    {
      name: 'webmars-store-v1',
      storage: createJSONStorage(() => localStorage),
      // Only persist a small allowlist. The simulator core state
      // (registers, memory, programCounter, console output) is NOT
      // persisted because the user expects a fresh run after refresh.
      partialize: (s) => ({
        source: s.source,
        files: s.files,
        activeFileId: s.activeFileId,
        editorFontSize: s.editorFontSize,
        themePreference: s.themePreference,  // new slice, see Bug 2.2
        currentSnippetId: s.currentSnippetId, // new slice, see Section 5
      }),
      version: 1,
```

```
    }
  )
)
```

Edge cases worth handling:

- If localStorage is full or unavailable (Safari private mode), Zustand's persist middleware logs a warning and falls through. The app still works, just without persistence. Acceptable.
- If the persisted payload is from an older schema, the version field triggers a migrate function. For v1 we just discard mismatches; later versions can write proper migrations.
- Storage size: even a 100 KB source file is well under localStorage's 5 MB cap. No quota issue at realistic sizes.

> **DESIGN NOTE**
> Once the account system is live (Section 5), authenticated users will also have their work saved server-side via the Snippets API. localStorage persistence is the safety net for both anonymous users and the gap between edits and the next save click.

2.2 Light mode shows a dark editor

[FRONTEND] **[MEDIUM]** Editor stays black while every panel around it turns white. Looks broken.

Where the bug lives

File: src/ui/CodeEditor.tsx, line 220

The Monaco theme is hardcoded:

```
<Editor
  height={size.h || '100%'}
  width={size.w || '100%'}
  language="mips"
  theme="webmars-dark"   // <-- always dark, even when app is in light mode
  value={source}
  // ...
```

Why it happens

Monaco maintains its own theme registry separate from the document's CSS. Setting Tailwind's dark-mode class on the body does nothing to the Monaco editor instance. The theme name passed to the Editor component is the only thing Monaco looHow to fix it

1. Add a themePreference slice to the Zustand store with values 'light', 'dark', or 'system'.
2. Define a second Monaco theme (webmars-light) in the same place that webmars-dark is currently defined.
3. In CodeEditor.tsx, replace the hardcoded theme prop with a value derived from themePreference.
4. Use a media query listener for 'system' so the editor follows the OS preference.

```
// Where webmars-dark is currently defined, add a sibling:
monaco.editor.defineTheme('webmars-light', {
  base: 'vs',
  inherit: true,
```

```
  rules: [
    { token: 'comment',         foreground: '6A737D', fontStyle: 'italic' },
    { token: 'keyword.mips',    foreground: '0033B3', fontStyle: 'bold' },
    { token: 'register.mips',   foreground: 'C25500' },
    { token: 'number.mips',     foreground: '098658' },
    { token: 'directive.mips',  foreground: '795E26' },
    { token: 'label.mips',      foreground: 'AF00DB' },
    { token: 'string',          foreground: 'A31515' },
  ],
  colors: {
    'editor.background':        '#FFFFFF',
    'editor.foreground':        '#1A1A1A',
    'editorLineNumber.foreground': '#A0A0A0',
    'editor.selectionBackground':  '#ADD6FF',
    'editor.lineHighlightBackground': '#F5F5F5',
  },
})

// In the Editor component:
const themePreference = useSimulator((s) => s.themePreference)  // 'light' | 'dark' |
'system'
const systemPrefersDark = useSystemColorScheme()  // a small hook
const effectiveTheme = themePreference === 'system'
  ? (systemPrefersDark ? 'webmars-dark' : 'webmars-light')
  : themePreference === 'light' ? 'webmars-light' : 'webmars-dark'

<Editor
  theme={effectiveTheme}
  // ...
/>
```

Add a theme toggle to SettingsDialog.tsx so users can pick light / dark / system. Persist the choice via the same persist middleware from Bug 2.1.

2.3 Anyone with a valid token can delete any snippet

[BACKEND] **[CRITICAL]** Any authenticated user can DELETE another user's snippet.

Where the bug lives

File: webmars-api/.../SnippetController.java, lines 25-28

```
@DeleteMapping("/{id}")
public void deleteSnippet(@PathVariable Long id){
     repository.deleteById(id);
}
```

There is no check that the authenticated user owns the snippet. Once accounts ship, this becomes a vandalism vector: an attacker registers an account, gets a token, and can wipe every snippet in the database by iterating ids.

How to fix it

Once snippets have an owner_id column (Section 5), the controller must verify ownership before delete:

```
@DeleteMapping("/{id}")
```

```
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable Long id, @AuthenticationPrincipal String username) {
    Snippet s = snippets.findById(id)
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
    if (!s.getOwner().getUsername().equals(username)) {
        // 404 not 403 - don't reveal the snippet exists to non-owners.
        throw new ResponseStatusException(HttpStatus.NOT_FOUND);
    }
    snippets.delete(s);
}
```

2.4 Wrong password returns HTTP 500

[BACKEND] **[MEDIUM]** The login endpoint can't tell the frontend whether the server is broken or the user typed the wrong password.

Where the bug lives

File: webmars-api/.../AuthController.java, lines 29-33

```
User found = userRepository.findByUsername(user.getUsername())
    .orElseThrow(() -> new RuntimeException("User not found"));
if (!passwordEncoder.matches(user.getPassword(), found.getPassword())) {
    throw new RuntimeException("Invalid Password");
}
```

Spring translates an uncaught RuntimeException into HTTP 500 Internal Server Error. The frontend has no way to distinguish 'user typed wrong password' from 'database is on fire'. Both look identical to the browser.

How to fix it

Throw ResponseStatusException(HttpStatus.UNAUTHORIZED) for credential failures. Also return the same message for both 'user not found' and 'bad password' so the endpoint does not leak which usernames exist:

```
@PostMapping("/login")
public Map<String, String> login(@Valid @RequestBody LoginRequest req) {
    User found = userRepository.findByUsername(req.username())
        .orElseThrow(() -> new ResponseStatusException(
            HttpStatus.UNAUTHORIZED, "Invalid credentials"));
    if (!passwordEncoder.matches(req.password(), found.getPassword())) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Invalid credentials");
    }
    return Map.of("token", jwtUtil.generateToken(found.getUsername()));
}
```

2.5 Database password and JWT signing key are committed to git

[BACKEND] **[CRITICAL]** Anyone with read access to the repository can sign forged tokens or connect to the production database.

What is exposed

File: webmars-api/src/main/resources/application.properties

```
spring.datasource.password=
```

File: webmars-api/src/main/java/com/webmars/webmars_api/JwtUtil.java

```
private final SecretKey key = Keys.hmacShaKeyFor(
    "webmars-super-secret-key-must-be-32-bytes!".getBytes()
);
```

How to fix it

5. Rotate the Postgres password in pgAdmin or via psql: ALTER USER postgres WITH PASSWORD 'newPasswordHere'.
6. Replace application.properties with environment-variable indirection (full snippet below).
7. Generate a fresh JWT secret with: openssl rand -base64 48
8. Refactor JwtUtil to read the secret from @Value("${JWT_SECRET}") and validate length.
9. Scrub git history of the old secrets: git filter-repo --replace-text passwords.txt — and force push.
10. Force any team member with a clone to re-clone after the history rewrite.

```
# application.properties (new version)
spring.application.name=webmars-api
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/webmars}
spring.datasource.username=${DB_USER:postgres}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=${JPA_DDL:update}
spring.jpa.show-sql=${JPA_SHOW_SQL:false}
jwt.secret=${JWT_SECRET}
```

REMINDER

A clean .gitignore does NOT solve this. The secrets are in past commits and visible via the git log. History must be rewritten. After the rewrite, every clone of the repo must be discarded and re-cloned.

2.6 Screen magnifier doesn't actually magnify

[FRONTEND] **[LOW]** The tool renders a box with the text 'cursor at x,y' instead of a magnified view.

Where the bug lives

File: src/ui/tools/ScreenMagnifier.tsx, lines 79-83

A comment in the file admits the implementation is a placeholder:

```
// Inline a clone via a CSS background-image-like trick is
// complex; use a simplified alternative: render an
// informational placeholder. The full page-clone approach
// requires html2canvas or postprocessing; out of scope for
// SA-15.
```

The PRD's section on out-of-scope also lists html2canvas-style screen magnification. That decision is now being reversed.

Fix: DOM-clone implementation

Rather than rasterizing the page with html2canvas (heavy dependency, breaks on Monaco's contenteditable surface), we clone the DOM element under the cursor into the loupe container and let the browser render it twice at a 2x CSS transform. Specifics in Section 7 (Zachary's track).

2.7 No user feedback when API calls fail

[FRONTEND] **[MEDIUM]** Failed save / login / load does nothing visible. Users assume the app froze.

How to fix it

Add a small toast notification system. Two options:

- Use sonner (npm install sonner) - 4KB, drop-in, dark/light theming built in.
- Build a tiny in-house toast using a Zustand slice and a fixed-position container - 30 lines of code, zero new dependencies.

Recommendation: go with sonner. The save in lines of code is not worth maintaining custom toast logic, and sonner is widely used and tiny.

2.8 No loading indicators for network calls

[FRONTEND] **[LOW]** Save buttons stay clickable during the request; clicking twice creates duplicate snippets.

How to fix it

Every async API call should set a loading state in the corresponding component. Disable the button and swap its label to a spinner or 'Saving...' text while the request is in flight. Pattern:

```
const [saving, setSaving] = useState(false)
async function onSave() {
  setSaving(true)
  try {
    const snippet = await api.saveSnippet({ title, code })
    toast.success(`Saved as ${snippet.title}`)
  } catch (err) {
    toast.error(err instanceof ApiError
      ? `Save failed: ${err.body}`
      : 'Save failed. Check your connection.')
  } finally {
    setSaving(false)
  }
}
<button disabled={saving} onClick={onSave}>
  {saving ? 'Saving...' : 'Save'}
</button>
```

2.9 Shared links don't load anything on a cold visit

[FRONTEND] **[BLOCKER for v1.2]** The share URL pattern doesn't exist yet. Visiting webmarsimulator.com/?snippet=42 loads the default Hello-MIPS example.

How to fix it

On app boot, check window.location.search for ?snippet=<id>. If present, call GET /snippets/{id} and load the result into the source slice. If the snippet is private and the user is not logged in or not the owner, the API returns 404 and the editor loads the default example with a toast: 'That snippet is private or does not exist.'

Extend the existing src/lib/router.ts (per the README it's a 30-line History API wrapper) to parse query params. Detail in Section 5.

2.10 JWT expires after 15 minutes

[BACKEND] **[LOW]** Users get silently logged out mid-session. The frontend has no refresh logic.

Where the bug lives

File: webmars-api/.../JwtUtil.java, line 19

```
.expiration(new Date(System.currentTimeMillis() + 1000 * 60 * 15))
```

How to fix it

Two options. Pick one for this sprint:

- Short-term: extend the token lifetime to 8 hours. Trivial change. Acceptable for a course project; users re-login once per work session.
- Proper fix: implement refresh tokens. Issue a short access token (15 min) plus a longer-lived refresh token stored in an HttpOnly cookie. Frontend silently refreshes when it gets a 401. Larger change, recommended for v1.3.

Recommendation for this sprint: extend to 8 hours. Note it in the v1.3 backlog as a follow-up.

2.11 No CORS configuration - frontend can't call the API

[BACKEND] **[BLOCKER]** Without CORS the frontend cannot fetch anything from a different origin. This is why no integration exists yet.

Where the bug lives

File: webmars-api/.../SecurityConfig.java

The SecurityFilterChain disables CSRF and configures route authorization but never enables CORS. Browser preflight requests fail with 'CORS policy: No Access-Control-Allow-Origin header'.

How to fix it

Add a CorsConfigurationSource bean and call .cors(Customizer.withDefaults()) on the HttpSecurity builder. Full snippet in Section 4.

3. Backend Architecture and Tech Stack

The backend (webmars-api) already exists. This section documents what's there, what we're adding, how to run it locally, and how it deploys. New team members should be able to clone, build, and run the API locally within thirty minutes of reading this section.

3.1 Current tech stack

Component	Choice	Purpose
Language	Java 21	Records (DTOs), pattern matching, modern language features.
Framework	Spring Boot 4.0.6	REST controllers, dependency injection, embedded Tomcat.
Build	Maven (mvnw wrapper)	No system-wide Maven install needed. Reproducible builds.
Database	PostgreSQL 18	Relational, ACID, free, well-supported by Spring Data.
ORM	Spring Data JPA + Hibernate	Repository interfaces, automatic query generation.
Auth	Spring Security + jjwt 0.12.6	JWT-based stateless authentication.
Password hashing	BCrypt	Adaptive cost factor (default 10 rounds).
Migrations	Hibernate ddl-auto=update (legacy)	Will replace with Flyway in this sprint.

3.2 What we are adding

- Flyway for explicit, versioned schema migrations. Migrations live in src/main/resources/db/migration as V1__init.sql, V2__add_ownership.sql, V3__runs_table.sql.
- spring-boot-starter-validation for request body validation via Jakarta annotations (@NotBlank, @Size, @Pattern).
- Bucket4j for rate limiting on the auth endpoints. Five login attempts per username per fifteen minutes, ten registrations per IP per hour.
- Testcontainers for integration tests against a real PostgreSQL instance in CI, not a mocked one.
- Environment-variable configuration via @Value injection. Secrets out of git.

3.3 Final project structure

```
webmars-api/
├── pom.xml                          # +validation, +flyway, +bucket4j, +testcontainers
├── .env.example                     # NEW - documents required env vars
├── .gitignore                       # MODIFY - ensure .env is excluded
├── src/main/java/com/webmars/webmars_api/
│   ├── WebmarsApiApplication.java
│   ├── SecurityConfig.java          # MODIFY - CORS, public GET, validation
│   ├── JwtUtil.java                 # MODIFY - env var secret, longer expiry
│   ├── JwtFilter.java
│   ├── AuthController.java          # MODIFY - DTOs, proper status codes
│   ├── User.java                    # MODIFY - @JsonIgnore on password
│   ├── UserRepository.java
│   ├── Snippet.java                 # MODIFY - owner FK, visibility, updated_at
│   ├── SnippetRepository.java       # MODIFY - findByOwnerId, findByVisibility
│   ├── SnippetController.java       # REWRITE - mine, public, update, owner check
│   ├── Run.java                     # NEW
│   ├── RunRepository.java           # NEW - includes most-run projection query
│   ├── RunController.java           # NEW
│   ├── GlobalExceptionHandler.java  # NEW - @RestControllerAdvice
│   ├── RateLimitFilter.java         # NEW - Bucket4j filter
│   └── dto/
│       ├── LoginRequest.java        # NEW
│       ├── RegisterRequest.java     # NEW
│       ├── UserResponse.java        # NEW
│       ├── SaveSnippetRequest.java  # NEW
│       ├── UpdateSnippetRequest.java # NEW
│       ├── SnippetResponse.java     # NEW
│       ├── LogRunRequest.java       # NEW
│       ├── RunResponse.java         # NEW
│       └── MostRunResponse.java     # NEW
├── src/main/resources/
│   ├── application.properties       # MODIFY - env var indirection
│   └── db/migration/
│       ├── V1__init.sql             # NEW - users, snippets baseline
│       ├── V2__snippet_ownership.sql # NEW - owner_id, visibility, updated_at
│       └── V3__runs.sql             # NEW - runs table
└── src/test/java/com/webmars/webmars_api/
    ├── WebmarsApiApplicationTests.java
    ├── AuthControllerTest.java      # NEW
    ├── SnippetControllerTest.java   # NEW
    ├── RunControllerTest.java       # NEW
    ├── JwtUtilTest.java             # NEW
    └── PostgresTestBase.java        # NEW - Testcontainers shared base
```

3.4 Final database schema

After all three migrations apply, the schema looks like:

```
-- USERS table
CREATE TABLE users (
    id          BIGSERIAL PRIMARY KEY,
    username    VARCHAR(32)  UNIQUE NOT NULL,
    password    VARCHAR(255) NOT NULL,          -- BCrypt hash
    created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
```

```
);

-- SNIPPETS table
CREATE TABLE snippets (
    id          BIGSERIAL PRIMARY KEY,
    title       VARCHAR(255),
    code        TEXT NOT NULL,
    owner_id    BIGINT REFERENCES users(id) ON DELETE CASCADE,
    visibility  VARCHAR(10) NOT NULL DEFAULT 'PRIVATE'
                CHECK (visibility IN ('PUBLIC', 'PRIVATE')),
    created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_snippets_owner_id   ON snippets(owner_id);
CREATE INDEX idx_snippets_visibility ON snippets(visibility) WHERE visibility = 'PUBLIC';

-- RUNS table
CREATE TABLE runs (
    id                   BIGSERIAL PRIMARY KEY,
    snippet_id           BIGINT NOT NULL REFERENCES snippets(id) ON DELETE CASCADE,
    user_id              BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    started_at           TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    duration_ms          INTEGER,
    instructions_executed INTEGER,
    exit_status          VARCHAR(20) CHECK (exit_status IN
                         ('completed','error','paused','aborted')),
    error_message        TEXT
);

CREATE INDEX idx_runs_user_started ON runs(user_id, started_at DESC);
CREATE INDEX idx_runs_snippet      ON runs(snippet_id);
```

3.5 Running the backend locally

These steps work on macOS, Linux, and Windows (PowerShell). Each developer does this once.

3.5.1 Prerequisites

- Java 21 (Eclipse Temurin recommended). Verify: java --version.
- PostgreSQL 18 running locally. macOS: brew install postgresql@18. Windows: installer from postgresql.org.
- Optional: pgAdmin 4 for a GUI. The psql CLI is enough.
- IntelliJ IDEA Community or VS Code with the Extension Pack for Java. Not required but it helps.

3.5.2 First-time setup

11. Clone the repo: git clone https://github.com/landclay715/webmars-api.git
12. Create the database: psql -U postgres -c 'CREATE DATABASE webmars;'
13. Copy the example env file: cp .env.example .env

14. Edit .env and fill in DB_PASSWORD (whatever your local postgres password is) and JWT_SECRET (run openssl rand -base64 48 to generate one).
15. Export the env vars in your shell (or use IntelliJ's run configuration env vars field): export $(grep -v '^#' .env | xargs)
16. Build: ./mvnw clean package -DskipTests
17. Run: ./mvnw spring-boot:run
18. Test it's alive: curl http://localhost:8080/snippets/1 - should return 404 not connection refused.

3.5.3 Verifying auth works end-to-end

```
# 1. Register a test user
curl -X POST http://localhost:8080/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"testuser","password":"password123"}'

# Expected: 200 with user info JSON, no password field

# 2. Login and capture the token
TOKEN=$(curl -s -X POST http://localhost:8080/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"testuser","password":"password123"}' \
  | jq -r '.token')
echo $TOKEN

# Expected: a JWT (three dot-separated base64 chunks)

# 3. Save a snippet
curl -X POST http://localhost:8080/snippets/save \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"title":"Test","code":"li $t0, 5\n","visibility":"PUBLIC"}'

# Expected: 200 with the saved snippet, including id and ownerUsername
```

3.6 Deployment plan

3.6.1 Backend on Render

19. Push the repo to GitHub. Make sure the secrets are scrubbed (see Bug 2.5).
20. Create a new Render Web Service from the GitHub repo. Build command: ./mvnw clean package -DskipTests. Start command: java -jar target/webmars-api-0.0.1-SNAPSHOT.jar.
21. Add a Render PostgreSQL service. Render gives you a DATABASE_URL automatically.
22. Set env vars on the Web Service: DB_URL=<from Render PG>, DB_USER=<from Render PG>, DB_PASSWORD=<from Render PG>, JWT_SECRET=<freshly generated>, JPA_DDL=validate (Flyway runs the migrations, JPA should only validate the schema matches the entities).
23. First deploy may take 8-12 minutes (Maven downloads, JVM startup). Subsequent deploys are faster.

24. Render free tier sleeps after 15 minutes of inactivity. First request after sleep takes about 30 seconds. Acceptable for a course project; upgrade to the paid tier if usage warrants.

3.6.2 Frontend on Vercel

The frontend already deploys to Vercel (vercel.json is in the repo). The only change needed:

25. In Vercel dashboard, add an environment variable: VITE_API_BASE=https://webmars-api.onrender.com (replace with your actual Render URL).
26. Redeploy. Vite bakes the env var into the production bundle at build time.
27. Add the Vercel production domain to the CORS allowed-origins list in SecurityConfig.java and redeploy the backend.

> **DEPLOYMENT GOTCHA**
> Render's free PostgreSQL tier expires after 90 days unless you pay. For a course project that ships in two weeks, this is fine. Set a calendar reminder to either upgrade or migrate before the 90-day mark.

4. Team Roles and Responsibilities

The three engineers have established specialties from the v1.0 build. This sprint sticks with those specialties so the learning curve stays low. Where two tracks intersect, the listed primary owner handles the work and the other reviews the PR.

4.1 Bryan Djenabia - Frontend, Integration, Deployment

Bryan owns everything users see and click. In this sprint that means the API integration layer, all new modals and panels, the persistence and theme bug fixes, error toasts, and the production deploys.

Primary deliverables

- src/lib/api.ts - the typed HTTP client used by everything else on the frontend.
- Auth slice in the Zustand store, plus AuthModal (login/register UI).
- Zustand persist middleware around the existing store (fixes refresh-loses-code).
- Monaco light theme + theme preference plumbing (fixes dark editor in light mode).
- ShareModal - opens after Save, shows the shareable URL with copy button.
- URL-on-load handler - parses ?snippet=<id> and hydrates the editor.
- MySnippetsDrawer - lists user's saved snippets, click to load.
- HistoryPanel - shows recent runs and most-run programs.
- Public/private toggle in the Save modal.
- Toast notification system (sonner integration).
- Loading states on every async button.
- Vercel deployment - env vars, production build verification.

4.2 Landon Clay - Backend, Assembler, Security

Landon owns the API and the assembler internals. In this sprint that means hardening the existing auth, the schema migrations, the new endpoints for accounts and history, and the rem pseudo-instruction in the assembler.

Primary deliverables

- Security hardening: rotate secrets, env-var indirection, ResponseStatusException for auth failures, @JsonIgnore on User.password, input validation via Jakarta annotations, rate limiting on /auth/**.
- CORS configuration in SecurityConfig.
- Flyway adoption and V1/V2/V3 migrations.
- Snippet entity update: owner FK, visibility enum, updated_at column.
- Run entity and repository (including the most-run JPQL projection).
- Rewritten SnippetController: /mine, /public, /save, PUT, DELETE with owner checks.
- New RunController: POST /runs, GET /runs/recent, GET /runs/most-run.
- GlobalExceptionHandler - @RestControllerAdvice mapping validation errors to JSON 400s.

- rem pseudo-instruction in the assembler: lexer mnemonic, parser signature, assembler expansion.
- Render deployment - service setup, env vars, Postgres provisioning.

4.3 Zachary Gass - Simulator, Tools, Testing

Zachary owns the simulator core and the in-app tools. In this sprint his work splits between the bitmap and magnifier polish, the run-logging hook that connects the simulator to the new RunController, and the bulk of the test coverage.

Primary deliverables

- Run logging hook - after every run/error/abort, POST a Run record. Skips if user is not logged in or snippet is unsaved.
- BitmapDisplay 8x8 and 16x16 grid options.
- ScreenMagnifier rewrite using DOM-clone + CSS transform (Option B from the original plan).
- data-magnify-region attributes on register table, memory panel, console, status bar.
- rem pseudo-instruction tests (assembler unit tests + runtime tests in src/tests/).
- Backend integration tests via Testcontainers - AuthController, SnippetController, RunController, JwtUtil.
- End-to-end manual QA pass before each deploy.
- Help dialog update with rem documentation.

4.4 How the three tracks meet

Three points of contact require coordination:

- rem instruction: Landon writes the assembler code, Zachary writes the tests. Same PR. Day 6.
- Run logging: Landon must finish RunController before Zachary can wire the hook. Landon's Day 6, Zachary's Day 7.
- Auth-aware UI: Bryan needs Landon's hardened /auth/login (proper status codes, JSON response) before he can wire the AuthModal cleanly. Landon's Day 2-3, Bryan's Day 4.

Aside from those three, the tracks are independent and run in parallel.

5. Two-Week Sprint Plan - Day by Day

Ten working days, three engineers, parallel tracks. Each day's commitments are sized so a developer can finish them in about three to four focused hours, leaving time for code review, classes, and the inevitable broken environment. Friday afternoons are deliberately lighter so the team can demo and integrate, and the two weekend days at the midpoint and end are buffer for whatever slipped.

DAILY RITUAL

Start each day with a 10-minute standup (in person or async in Slack/Discord): yesterday, today, blockers. End each day by pushing what you have to a branch and opening a PR if there's anything ready for review. Do not let work-in-progress sit on a local machine for more than 24 hours - if you get sick or your laptop dies, the team can pick it up.

Conventions used below

- Branch name: feat/<sprint-day>-<task-slug>. Example: feat/d2-cors-config. One PR per branch.
- PR title format follows Conventional Commits: feat:, fix:, chore:, docs:, test:, refactor:.
- Every PR needs at least one reviewer from a different engineer. The reviewer pulls the branch locally and tests before approving.
- Tests required for any change to assembler, controllers, or store slices. UI work is exempt but should be hand-tested.
- All time estimates are working hours of focused coding, excluding meetings, review wait, debugging environment issues.

Week 1 - Foundations and Bug Fixes

Goal: by end of week one, the security holes are closed, the persistence and theme bugs are gone, the schema is migrated to support v1.2, and rem + 8x8 bitmap have shipped. No new account features are live yet but the foundations exist.

Day 1 (Monday) - Kickoff and Security Sprint

MILESTONE

By end of day: all secrets out of the repo, JWT secret rotated, every clone re-pulled.

Bryan	• Read this document end-to-end. Confirm Vercel access and admin rights on the WebMARS Vercel project. • Create the .env.local.example file in the frontend repo with VITE_API_BASE=http://localhost:8080. • Add sonner to package.json: npm install sonner. Add the Toaster component near the root of App.tsx. • Branch feat/d1-toast-system, open PR. Small confidence-building win. • Pair with Landon for 30 min: review the existing AuthController and JwtUtil so you understand the contract before integration starts on Day 4.
Landon	• Rotate the production Postgres password in pgAdmin: ALTER USER postgres WITH PASSWORD <new>. • Replace application.properties with env-var indirection (see Section 3.5 of this doc). • Create a fresh .env file locally with DB_PASSWORD and JWT_SECRET. Add .env to .gitignore. Commit a .env.example. • Refactor JwtUtil.java to read JWT_SECRET via @Value injection, validate it is at least 32 bytes, throw on missing. • Generate a fresh JWT secret: openssl rand -base64 48. Save it to your .env, NOT to git. • Scrub git history of old secrets using git filter-repo. Force-push. Notify team in chat to re-clone. • Verify with curl that auth still works after the rotation.
Zachary	• Pull latest, confirm the existing simulator and assembler tests still pass: npm test. • Branch feat/d1-bitmap-8x8. Edit src/ui/tools/BitmapDisplay.tsx line 16: DIMENSION_OPTIONS = [8, 16, 32, 64, 128, 256]. • Hand-test the 8x8 and 16x16 grid options in the dev server. Confirm the canvas renders correctly and the footer pixel count is right. • Open PR. Should be 1-line code change + a short README note in the PR description. • Read the existing src/ui/tools/ScreenMagnifier.tsx and the comment block on lines 76-83. Sketch the DOM-clone approach in a doc comment so Day 3 starts with a clear plan.

Day 2 (Tuesday) - Backend Hardening, Frontend Foundations

Bryan	• Branch feat/d2-persist-store. Add Zustand persist middleware around useSimulator (see Bug 2.1 in Section 2). • Define the partialize allowlist: source, files, activeFileId, editorFontSize. Do NOT persist simulator state. • Hand-test: open the dev server, edit code, refresh, confirm code is preserved. • Hand-test: also confirm registers and memory ARE reset on refresh (state should be fresh, code should not). • Open PR. Include before/after screenshots in the PR description. • Stub out src/lib/api.ts with the skeleton: BASE constant, authHeader helper, ApiError class, empty function signatures for

	saveSnippet/getSnippet/deleteSnippet/login/register. Real implementations come Day 3-4.
Landon	• Branch feat/d2-cors-config. Add CorsConfigurationSource bean to SecurityConfig.java and .cors(Customizer.withDefaults()) to the filter chain. See Section 4.1 of this doc for the full snippet. • Add allowed origins for both dev (localhost:5173 / 5174) and production (https://webmarsimulator.com). • Hand-test with a quick fetch from the dev frontend: confirm no CORS errors in browser console. • Branch feat/d2-validation. Add spring-boot-starter-validation to pom.xml. Run ./mvnw clean compile to confirm it pulls correctly. • Create RegisterRequest and LoginRequest record DTOs in com.webmars.webmars_api.dto package with Jakarta validation annotations. • Open both PRs separately so they can be reviewed and merged independently.
Zachary	• Branch feat/d2-magnifier-region-tags. Add data-magnify-region attribute to the obvious magnifiable elements: register table wrapper, memory panel wrapper, console output, status bar. • Search the UI files for these wrappers (most are in src/ui/RegisterTable.tsx, src/ui/MemoryPanel.tsx, src/ui/ConsolePanel.tsx, src/ui/StatusBar.tsx). • This is prep work for Day 3's magnifier rewrite - no behavior change today. • Open PR. Tiny change but it isolates the markup change from the logic change tomorrow. • Begin sketching the magnifier rewrite. Write a draft of the new useEffect that picks the element under the cursor and clones it. Save as a doc comment in ScreenMagnifier.tsx for tomorrow.

Day 3 (Wednesday) - Auth UX, Light Theme, Magnifier

Bryan	• Branch feat/d3-monaco-light-theme. Define webmars-light Monaco theme alongside the existing webmars-dark. Full theme JSON in Section 2.2. • Add a themePreference slice to the Zustand store ('light' | 'dark' | 'system', default 'dark' for backwards compatibility). • Replace the hardcoded theme="webmars-dark" in CodeEditor.tsx with the derived value. • Add a useSystemColorScheme hook (10-line wrapper over matchMedia('(prefers-color-scheme: dark)')). • Wire a theme toggle into SettingsDialog.tsx (radio group: Light / Dark / System). • Hand-test: switch to light mode in the settings, confirm the editor lightens immediately. Switch back, confirm dark returns. • Hand-test: 'System' option follows OS preference - toggle dark mode in macOS / Windows settings.
Landon	• Branch feat/d3-auth-status-codes. Rewrite AuthController to use @Valid RegisterRequest and LoginRequest, return JSON response objects, throw ResponseStatusException for 401/409 conditions.

	• Add @JsonIgnore on User.password. • Create UserResponse record DTO. AuthController.register now returns UserResponse, not the raw User entity. • Add a GlobalExceptionHandler @RestControllerAdvice that turns MethodArgumentNotValidException into JSON 400 with a field->error map. • Hand-test: send a register request with username='ab' (too short), expect 400 with a clear error message naming the username field. • Hand-test: login with a wrong password, expect 401 with body {"error": "Invalid credentials"}.
Zachary	• Branch feat/d3-magnifier-rewrite. Implement DOM-clone magnifier per Section 7's Zachary plan. • Pick the element under the cursor via document.elementFromPoint(x, y).closest('[data-magnify-region]'). If null, show a hint 'Hover a panel to magnify'. • Clone the element with cloneNode(true) into the loupe container. Re-clone only when the target changes, not on every mousemove. • Apply transform: scale(2) translate(...) to keep the cursor position centered. • Hand-test: turn on the magnifier, hover the register table, confirm you can read each register's value enlarged. • Edge case: Monaco editor is not tagged data-magnify-region (its DOM is too dynamic for stable clones). Magnifier over the editor shows the hint instead.

Day 4 (Thursday) - First Integration - Login Works End-to-End

MILESTONE
By end of day: a user can register and log in through the UI against the real backend.

Bryan	• Branch feat/d4-auth-modal. Build the AuthModal component (two tabs: Login, Register). Use the existing dialog primitives from SettingsDialog as a reference for style consistency. • Wire the modal to api.login and api.register. On success, store the token in localStorage (key: 'webmars.token') and update the auth slice in the store. • Add a 'Sign in' button to the top toolbar (next to the existing buttons). When authenticated, swap it for the username dropdown with a 'Log out' option. • Add loading states (saving / signing in spinner) and toast notifications for success and failure. • End-to-end test against Landon's running backend: register a fresh user, see the welcome toast, log out, log back in. • Open PR. Include a 10-second screen recording of the auth flow in the PR description.

Landon	• Branch feat/d4-flyway-setup. Add flyway-core to pom.xml. Create src/main/resources/db/migration/. • Write V1__init.sql. Capture the current users and snippets schema as-is. This becomes the baseline. • Set spring.jpa.hibernate.ddl-auto to validate in application.properties (Flyway runs migrations on boot, JPA only validates). • Test that the app still boots and existing data is intact. • Be available for Bryan to consult during the auth-modal integration. Likely a couple of small backend tweaks will surface (CORS preflight on a header, status code on duplicate username, etc.). Fix in-place if quick.
Zachary	• Polish day on the magnifier: handle edge cases (cursor leaves viewport, target element unmounts, target element is the loupe itself). • Add an aria-label and keyboard shortcut (Escape closes - already there but verify). • Open the magnifier PR with a screen recording. • Branch feat/d4-rem-lexer. Add 'rem' to MNEMONICS in lexer.ts (one-line change). Add rem to OPERAND_SIGNATURES in parser.ts (also one line). • Write the first failing test in src/tests/assembler.test.ts: rem $t0, $t1, $t2 should produce two machine words. Confirm test fails because assembler.ts doesn't handle rem yet. • Hand off the implementation to Landon for Day 5 (Landon owns the assembler logic per his specialty).

Day 5 (Friday) - Snippet Save and Share - First Real Feature

MILESTONE By end of day: a logged-in user can save a snippet and copy a shareable link.

Bryan	• Branch feat/d5-save-and-share. Wire api.saveSnippet to a 'Save to Account' button (visible only when logged in). • Build the ShareModal: after successful save, show the snippet ID and a copy-to-clipboard link in the format https://webmarsimulator.com/?snippet=<id>. • Add public/private toggle to the Save dialog (defaults to PRIVATE). • Loading state on the Save button. Toast on success and failure. • Extend src/lib/router.ts to parse ?snippet=<id> on initial load. If present, call api.getSnippet(id), set the source in the store, update the document title to the snippet name. • Hand-test: save a snippet, copy the link, open in a new incognito tab. If snippet is public, it loads. If private, it 404s (snippet doesn't exist message).

Landon	• Branch feat/d5-rem-assembler. Implement the rem case in assembler.ts (the case block in Section 7 of this doc). Make Zachary's failing test pass. • Run npm test in the frontend repo to confirm. Push. • Branch feat/d5-v2-migration. Write V2__snippet_ownership.sql: ALTER TABLE snippets ADD COLUMN owner_id, visibility, updated_at. • Update the Snippet.java entity: add @ManyToOne owner field, Visibility enum, updated_at column with @PreUpdate. • Update SnippetRepository with findByOwnerIdOrderByUpdatedAtDesc and findByVisibility. • Update SnippetController.save to set owner = current authenticated user. • Run the migration on local DB, hand-test that save still works and the snippet now has an owner.
Zachary	• Once Landon's rem implementation lands, verify all three rem tests pass: expansion test, positive remainder, negative dividend. • Add a fourth test: rem by zero produces undefined behavior (currently MIPS hardware traps; assembler should pass through; runtime behavior is whatever the simulator already does for div by zero). • Open the rem-tests PR. • Update HelpDialog.tsx with the rem entry in the Pseudo-Instructions section. • Friday short day: rest of afternoon is buffer / catch-up. If on schedule, start reading about Testcontainers (Section 8) for next week's tests.

END OF WEEK 1 CHECK-IN

Schedule a 30-minute sync Friday afternoon. Each engineer demos their merged PRs. If anyone is behind, redistribute work for Week 2. Update the team Discord/Slack with a status post.

Week 2 - Features, Polish, Deployment

Goal: by end of week two, accounts and run history are fully working end-to-end, the magnifier and bitmap fixes are merged, both apps are deployed to production, and the team has done a manual QA pass against the live site.

Day 6 (Monday) - Run History Schema and Controller

Bryan	• Branch feat/d6-snippets-mine. Build the MySnippetsDrawer component. List snippets via api.getMySnippets (calls GET /snippets/mine). • Each row shows title, last-updated timestamp, visibility badge (public / private icon). Click to load snippet into editor. • Add edit-in-place for the title and a 'Delete' button per row with a confirm dialog. • Add a 'New Snippet' button that clears the editor and unsets currentSnippetId (so save creates a fresh row).

	• Hand-test against Landon's running backend with 3-4 snippets. • Confirm the persist middleware still saves currentSnippetId across refreshes so the editor remembers which snippet you were on.
Landon	• Branch feat/d6-runs-schema-and-entity. Write V3__runs.sql migration. • Create Run.java entity with snippet (ManyToOne), user (ManyToOne), startedAt, durationMs, instructionsExecuted, exitStatus enum, errorMessage. • Create RunRepository with findByUserIdOrderByStartedAtDesc and the most-run JPQL projection (see Section 7 Landon plan for the exact JPQL). • Hand-test: insert a few runs via SQL, query the projection from a Spring test or DB GUI. • Branch feat/d6-snippets-controller-rewrite. Rewrite SnippetController: get with visibility gating, /mine, /public paginated, /save sets owner, PUT update, DELETE with owner check. • Update SecurityConfig to permit GET /snippets/* but still authenticate PUT/POST/DELETE.
Zachary	• Branch feat/d6-run-logging-hook. In useSimulator, find the place where a run completes (likely a finalizeRun or onRunEnd action). • After successful run completion (and on error / abort), if the user is authenticated AND there is a currentSnippetId, call api.logRun({ snippetId, durationMs, instructionsExecuted, exitStatus }). • Wrap in a try/catch - run logging is fire-and-forget; do NOT block the simulator if the API is down. Log to console on failure. • Hand-test: not yet possible because Landon's RunController lands Day 7. For now, mock the api.logRun function and confirm it's called with the right payload.

Day 7 (Tuesday) - RunController + Public Snippet Feed

Bryan	• Branch feat/d7-public-feed. Build a simple public snippets browser (modal or new tab in the right inspector). • Calls api.getPublicSnippets(page, size). Shows title, owner username, last-updated, click-to-load. • Pagination controls at the bottom: Previous, Next, current page number. • Add a 'See public snippets' link in the empty-state of MySnippetsDrawer so anonymous users have a way to discover content. • Once Zachary's run logging lands, verify end-to-end: save a snippet, run it, refresh, check the runs table via psql or pgAdmin.
Landon	• Branch feat/d7-run-controller. Build RunController: POST /runs (log a run), GET /runs/recent, GET /runs/most-run. • POST /runs validates the LogRunRequest, verifies the snippet exists, sets user from JWT principal (not from request body - never trust client-claimed user IDs). • Add corresponding DTOs: LogRunRequest, RunResponse, MostRunResponse. • Hand-test all three endpoints with curl. Save a snippet, log 5 runs, confirm /recent returns 5 and /most-run shows the snippet with count=5.

Zachary	• Branch feat/d7-integration-tests. Add Testcontainers to pom.xml: testcontainers-postgresql, testcontainers-junit-jupiter. • Create PostgresTestBase with the @Testcontainers and @DynamicPropertySource setup so all integration tests share the same Postgres instance. • Write AuthControllerTest: register success, register duplicate username (409 expected), login success, login wrong password (401), login unknown user (401).

Day 8 (Wednesday) - History Panel and More Tests

Bryan	• Branch feat/d8-history-panel. Build the HistoryPanel as a new tab in the right inspector (next to Registers, Memory, etc.). • Two sections: 'Recent Runs' (api.getRecentRuns) and 'Most Run' (api.getMostRunPrograms). • Recent Runs: list of runs with snippet title, timestamp, duration, exit status (color-coded badge). • Most Run: list of snippets with run count and last run timestamp. • Click a run or most-run row -> loads the snippet into the editor. • Empty state for new users: 'Run a saved snippet to see your history here.'
Landon	• Branch feat/d8-rate-limiting. Add Bucket4j to pom.xml. Implement RateLimitFilter that intercepts /auth/** and limits per-username for login and per-IP for register. • On rate limit exceeded, return 429 Too Many Requests with a Retry-After header. • Hand-test: send 6 wrong-password attempts in a row; the 6th returns 429. • Branch feat/d8-jwt-expiry. Bump JWT expiry from 15 minutes to 8 hours in JwtUtil.java (one-line change: 1000 * 60 * 15 -> 1000 * 60 * 60 * 8). • Update the README with a v1.3 note: refresh tokens are a follow-up.
Zachary	• Branch feat/d8-more-integration-tests. Write SnippetControllerTest: owner can read private, non-owner gets 404 on private, anyone gets 200 on public, non-owner gets 404 on delete, owner can update. • Write RunControllerTest: log run sets user from JWT (not from body), recent and most-run are scoped to the authenticated user. • Write JwtUtilTest: round-trip a token, reject tampered token, reject expired token, reject token signed with different secret. • If all tests pass: run ./mvnw verify and check the test report. Should have ~25 integration tests passing.

Day 9 (Thursday) - Polish Day and Edge Cases

Bryan	• Branch feat/d9-polish. Empty states: every new panel needs an empty state. MySnippetsDrawer says 'Save your first snippet to see it here.' HistoryPanel

	says 'Run a saved snippet to see your history.' PublicFeed says 'No public snippets yet. Be the first.' • Confirm dialogs: deleting a snippet requires a confirm. 'Delete <title>? This cannot be undone.' [Cancel] [Delete]. • Keyboard accessibility: AuthModal, ShareModal, and confirm dialogs trap focus, close on Escape, submit on Enter. • Browser back button: navigating from /?snippet=12 to / and back should restore the snippet 12 state. Currently history.pushState is used; verify popstate restores state. • Mobile spot-check on a phone or via DevTools device emulation. The new modals should be usable below 768px width.
Landon	• Branch feat/d9-error-handling-polish. Review GlobalExceptionHandler. Make sure every Spring exception type maps to a JSON response, not a stack trace: • - MethodArgumentNotValidException -> 400 { field: message } • - ResponseStatusException -> the declared status with the reason • - DataIntegrityViolationException (e.g. duplicate username) -> 409 { error: '...' } • - Anything else -> 500 with a generic message (don't leak the stack trace) • Branch feat/d9-readme-update. Rewrite both repos' READMEs to reflect the v1.2 surface. New endpoints, env vars, deployment instructions, demo gif if available.
Zachary	• Branch feat/d9-help-dialog. Update HelpDialog.tsx with: the new rem entry, the screen magnifier description (mention which panels are magnifiable), the bitmap 8x8/16x16 options, the new keyboard shortcut for theme toggle if one exists. • Manual QA round 1. Use the checklist in Section 8 of this doc. Document any bugs found in Discord/Slack with screenshots. • If bugs found, triage with the team: must-fix-before-deploy vs known-issues-document.

Day 10 (Friday) - Deploy and Demo

MILESTONE By end of day: production is live at webmarsimulator.com talking to webmars-api on Render.	
Bryan	• Final pre-deploy checklist (frontend): npm run lint clean, npm run typecheck clean, npm test clean. • Update VITE_API_BASE in Vercel to the Render URL. • Trigger production deploy from Vercel dashboard. Watch the build log.

	• Once live, smoke test on the production URL: register, save a public snippet, copy share link, open in another browser, run, check history. All in production. • If smoke test passes, post in team chat with the URL and a screenshot. We are live.
Landon	• Final pre-deploy checklist (backend): ./mvnw verify all green, no warnings, no SQL show-sql logs in prod profile. • Create the Render Web Service. Configure env vars: DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET (fresh one - do NOT reuse local), JPA_DDL=validate. • Provision Render PostgreSQL. Note: free tier expires after 90 days. • Run Flyway migrations on the production DB (Spring Boot runs them on boot automatically). • Update SecurityConfig allowed-origins to include the Vercel production domain (https://www.webmarsimulator.com and https://webmarsimulator.com). • Coordinate with Bryan on the smoke test.
Zachary	• Manual QA round 2: against the LIVE production site. Repeat the Section 8 checklist. • File bugs as GitHub Issues with screenshots / repro steps for anything caught. • Triage with the team: hotfix today, hotfix Saturday morning, or document as known issue for v1.3. • Record a 2-minute demo video for whatever channel you use (course Slack, GitHub release page, README hero gif). Show: editor -> save -> share -> open in incognito -> run -> see history. • End-of-sprint retrospective notes: what went well, what to do differently next time. Five minutes per engineer.

WEEKEND BUFFER

Saturday and Sunday after the deploy are buffer for any production hotfix that the Friday smoke test surfaces. If everything is clean, the weekend is rest. If anything is broken, you have two days to fix it before users notice on Monday.

5.1 If a day slips - the decision tree

Things will slip. Here is how to decide what gets cut without breaking the sprint:

28. If Bryan slips on the auth modal: keep the backend on schedule. Save and Share can be exposed for ANY user (no auth) on a public-only basis as a fallback. Worse UX, still works.
29. If Landon slips on RunController: ship without run history this sprint. The /runs endpoints become v1.3. Zachary skips Day 7 integration test work and moves to UI polish to help Bryan.
30. If Zachary slips on the magnifier rewrite: ship with the placeholder still in place. The magnifier is the lowest-priority tool fix and the existing stub at least doesn't crash.

31. If two people slip simultaneously: pull the entire public-feed feature (Day 7 Bryan / Day 6-7 Landon). Public snippets still work via direct link; just no browse-by-public UI.

The non-negotiable cuts (must ship): security fixes, refresh persistence, light-mode bug, auth working end-to-end. Everything else can flex.

6. Bryan - Detailed Task Breakdown

This section is Bryan's reference for every task assigned to him. Each item has the exact files to touch, the code patterns to use, and the acceptance criteria for the PR review. Read this section once before starting Day 1 and refer back as needed.

6.1 The API client - src/lib/api.ts

Goal: a single typed HTTP module that every part of the UI uses for talking to the backend. No raw fetch calls anywhere else in src/. This makes the auth header, error handling, and base URL centrally controllable.

Acceptance criteria

- Every function in the module returns a typed result or throws an ApiError.
- Auth header is attached automatically when a token exists in localStorage.
- Base URL is environment-configurable via VITE_API_BASE.
- 404, 401, 409, 500 all surface to the caller in a distinguishable way (via ApiError.status).

Full skeleton

```
// src/lib/api.ts
const BASE = import.meta.env.VITE_API_BASE ?? 'http://localhost:8080'

export class ApiError extends Error {
  constructor(public status: number, public body: string) {
    super(`HTTP ${status}: ${body || '(no body)'}`)
  }
}

function authHeader(): HeadersInit {
  const t = localStorage.getItem('webmars.token')
  return t ? { Authorization: `Bearer ${t}` } : {}
}

async function request<T>(path: string, init?: RequestInit): Promise<T> {
  const res = await fetch(`${BASE}${path}`, {
    ...init,
    headers: {
      'Content-Type': 'application/json',
      ...authHeader(),
      ...(init?.headers || {}),
    },
  })
  if (!res.ok) {
    const body = await res.text().catch(() => '')
    throw new ApiError(res.status, body)
  }
  if (res.status === 204) return undefined as T
  return res.json() as Promise<T>
}

// ── Auth ──────────────────────────────────────────────────────────────
```

```
export interface UserResponse { id: number; username: string; createdAt: string }
export interface LoginResponse { token: string }

export const register = (username: string, password: string) =>
  request<UserResponse>('/auth/register', {
    method: 'POST',
    body: JSON.stringify({ username, password }),
  })

export const login = (username: string, password: string) =>
  request<LoginResponse>('/auth/login', {
    method: 'POST',
    body: JSON.stringify({ username, password }),
  })

// ── Snippets ──────────────────────────────────────────────────────────────
export type Visibility = 'PUBLIC' | 'PRIVATE'

export interface Snippet {
  id: number
  title: string
  code: string
  ownerUsername: string | null
  visibility: Visibility
  createdAt: string
  updatedAt: string
}

export const getSnippet      = (id: number) => request<Snippet>(`/snippets/${id}`)
export const getMySnippets   = ()           => request<Snippet[]>('/snippets/mine')
export const getPublicSnippets = (page=0, size=20) =>
  request<{content: Snippet[], totalPages: number, number:
number}>(`/snippets/public?page=${page}&size=${size}`)

export const saveSnippet = (body: { title: string; code: string; visibility?: Visibility
}) =>
  request<Snippet>('/snippets/save', { method: 'POST', body: JSON.stringify(body) })

export const updateSnippet = (id: number, body: Partial<{ title: string; code: string;
visibility: Visibility }>) =>
  request<Snippet>(`/snippets/${id}`, { method: 'PUT', body: JSON.stringify(body) })

export const deleteSnippet = (id: number) =>
  request<void>(`/snippets/${id}`, { method: 'DELETE' })

// ── Runs ──────────────────────────────────────────────────────────────────
export type ExitStatus = 'COMPLETED' | 'ERROR' | 'PAUSED' | 'ABORTED'

export interface Run {
  id: number
  snippetId: number
  snippetTitle: string
  startedAt: string
  durationMs: number | null
  instructionsExecuted: number | null
  exitStatus: ExitStatus
}

export interface MostRun {
  snippetId: number
```

```
  title: string
  runCount: number
  lastRun: string
}

export const logRun = (body: {
  snippetId: number
  durationMs?: number
  instructionsExecuted?: number
  exitStatus: ExitStatus
  errorMessage?: string
}) => request<Run>('/runs', { method: 'POST', body: JSON.stringify(body) })

export const getRecentRuns = (limit = 20) =>
  request<Run[]>(`/runs/recent?limit=${limit}`)

export const getMostRunPrograms = (limit = 10) =>
  request<MostRun[]>(`/runs/most-run?limit=${limit}`)
```

6.2 Zustand persist - fixing refresh-loses-code

Goal: editor source survives a page refresh. Simulator state (registers, memory, PC) does NOT (those should be fresh).

Acceptance criteria

- After editing code and pressing F5, the same code is in the editor on reload.
- After running a program, refreshing the page leaves all registers at zero (state was NOT persisted).
- Multiple tabs do not race-condition each other's writes (last-write-wins is acceptable).
- Disabling localStorage (private browsing) does not crash the app - it falls back to in-memory only.

Implementation

Wrap the store creator in Zustand's persist middleware. Use partialize to whitelist only the slices that should survive a refresh:

```
// src/hooks/useSimulator.ts (top of file)
import { create } from 'zustand'
import { persist, createJSONStorage } from 'zustand/middleware'

export const useSimulator = create<SimulatorStore>()(
  persist(
    (set, get) => ({
      // ... existing store body unchanged ...
    }),
    {
      name: 'webmars-store-v1',
      storage: createJSONStorage(() => localStorage),
      partialize: (s) => ({
        // Editor content + file tabs
        source:         s.source,
        files:          s.files,
```

```
        activeFileId:   s.activeFileId,
        // User preferences
        editorFontSize: s.editorFontSize,
        themePreference: s.themePreference,
        // Snippet metadata (so reload knows what snippet you're on)
        currentSnippetId: s.currentSnippetId,
        currentSnippetVisibility: s.currentSnippetVisibility,
      }),
      version: 1,
      // If the persisted shape changes later, add a migrate function here.
    }
  )
)
```

WHY NOT PERSIST EVERYTHING

Persisting the simulator core state (registers, memory, breakpoints, programCounter, instructionsExecuted) would surprise users on refresh - they expect a fresh run. It would also bloat localStorage every reload. The partialize allowlist keeps the persisted payload small and predictable.

6.3 Light theme for Monaco - SettingsDialog wire-up

Goal: switching the app's theme preference also switches the editor. Three options: light, dark, system.

Acceptance criteria

- User can pick Light / Dark / System in SettingsDialog.
- Setting persists across refresh (already covered by 6.2 if themePreference is in the allowlist).
- 'System' follows the OS preference and updates live when the OS theme changes.
- On the system option, the Tailwind dark-mode class also toggles to match.

The useSystemColorScheme hook

```
// src/hooks/useSystemColorScheme.ts
import { useEffect, useState } from 'react'

export function useSystemColorScheme(): 'light' | 'dark' {
  const get = () =>
    typeof window !== 'undefined' && window.matchMedia('(prefers-color-scheme:
dark)').matches
      ? 'dark' : 'light'

  const [scheme, setScheme] = useState<'light' | 'dark'>(get)

  useEffect(() => {
    const mq = window.matchMedia('(prefers-color-scheme: dark)')
    const onChange = () => setScheme(mq.matches ? 'dark' : 'light')
    mq.addEventListener('change', onChange)
    return () => mq.removeEventListener('change', onChange)
  }, [])
```

```
  return scheme
}
```

Wiring it up in CodeEditor.tsx

```
// Near the top of CodeEditor.tsx
const themePreference = useSimulator((s) => s.themePreference)     // 'light' | 'dark' |
'system'
const systemScheme = useSystemColorScheme()                         // 'light' | 'dark'
const effectiveTheme =
  themePreference === 'system'
    ? (systemScheme === 'dark' ? 'webmars-dark' : 'webmars-light')
    : themePreference === 'light' ? 'webmars-light' : 'webmars-dark'

// Replace the hardcoded theme="webmars-dark" prop:
<Editor
  theme={effectiveTheme}
  // ... rest unchanged ...
/>
```

Wiring the rest of the app to match

Tailwind dark mode is class-based in the current setup. Add an effect at the App.tsx level to toggle the html class:

```
// src/App.tsx (or wherever the root component lives)
const themePreference = useSimulator((s) => s.themePreference)
const systemScheme = useSystemColorScheme()
const isDark = themePreference === 'dark' ||
               (themePreference === 'system' && systemScheme === 'dark')

useEffect(() => {
  document.documentElement.classList.toggle('dark', isDark)
}, [isDark])
```

6.4 AuthModal - login and register UI

Goal: a single dialog with tabs for Login and Register. Submits to the backend, stores the token, updates the auth slice.

State shape additions to the store

```
interface AuthSlice {
  authToken: string | null
  authUsername: string | null
  setAuth: (token: string, username: string) => void
  clearAuth: () => void
}

// In the store creator:
authToken: localStorage.getItem('webmars.token'),
authUsername: localStorage.getItem('webmars.username'),
setAuth: (token, username) => {
```

```
  localStorage.setItem('webmars.token', token)
  localStorage.setItem('webmars.username', username)
  set({ authToken: token, authUsername: username })
},
clearAuth: () => {
  localStorage.removeItem('webmars.token')
  localStorage.removeItem('webmars.username')
  set({ authToken: null, authUsername: null })
},
```

Component sketch

```
// src/ui/AuthModal.tsx
import { useState } from 'react'
import { toast } from 'sonner'
import * as api from '@/lib/api'
import { useSimulator } from '@/hooks/useSimulator'

export function AuthModal({ open, onClose }: { open: boolean; onClose: () => void }) {
  const [mode, setMode] = useState<'login' | 'register'>('login')
  const [username, setUsername] = useState('')
  const [password, setPassword] = useState('')
  const [busy, setBusy] = useState(false)
  const setAuth = useSimulator((s) => s.setAuth)

  if (!open) return null

  async function submit() {
    setBusy(true)
    try {
      if (mode === 'register') {
        await api.register(username, password)
        toast.success('Account created. Logging you in...')
      }
      const { token } = await api.login(username, password)
      setAuth(token, username)
      toast.success(`Signed in as ${username}`)
      onClose()
    } catch (err) {
      const msg = err instanceof api.ApiError
        ? (err.status === 401 ? 'Wrong username or password.' :
           err.status === 409 ? 'That username is taken.' :
           err.status === 429 ? 'Too many attempts. Try again in a few minutes.' :
           'Something went wrong. Please try again.')
        : 'Network error. Check your connection.'
      toast.error(msg)
    } finally {
      setBusy(false)
    }
  }

  return (
    /* dialog markup mirroring SettingsDialog styling.
       Tab strip switches mode. Input fields for username and password.
       Submit button label changes based on mode. Disabled while busy. */
  )
}
```

6.5 ShareModal - save and copy link

Goal: after the user clicks 'Save to Account' for the first time on a snippet, show them the URL they can share. Subsequent saves update the existing snippet in place.

Acceptance criteria

- If currentSnippetId is null, Save creates a new snippet (POST /snippets/save), assigns id, opens ShareModal.
- If currentSnippetId is set, Save updates that snippet (PUT /snippets/{id}). No modal.
- ShareModal shows the URL https://<host>/?snippet=<id> with a Copy button.
- Copy button uses navigator.clipboard.writeText. Toast 'Copied to clipboard' on success.
- Modal shows the visibility (PUBLIC or PRIVATE) and a toggle to change it. Toggling triggers a PUT.

6.6 Loading shared snippets on app boot

Goal: visiting webmarsimulator.com/?snippet=42 loads snippet 42 into the editor instead of the default example.

Implementation

```
// src/lib/router.ts (extend the existing wrapper)
export function getSnippetIdFromUrl(): number | null {
  const params = new URLSearchParams(window.location.search)
  const raw = params.get('snippet')
  if (!raw) return null
  const id = parseInt(raw, 10)
  return Number.isFinite(id) && id > 0 ? id : null
}

export function setSnippetIdInUrl(id: number | null): void {
  const url = new URL(window.location.href)
  if (id === null) url.searchParams.delete('snippet')
  else             url.searchParams.set('snippet', String(id))
  window.history.replaceState({}, '', url.toString())
}
```

```
// In App.tsx
useEffect(() => {
  const id = getSnippetIdFromUrl()
  if (id === null) return
  api.getSnippet(id)
    .then(snippet => {
      useSimulator.setState({
        source: snippet.code,
        currentSnippetId: snippet.id,
        currentSnippetVisibility: snippet.visibility,
      })
      document.title = `${snippet.title} - WebMARS`
    })
    .catch(err => {
      if (err instanceof api.ApiError && err.status === 404) {
```

```
        toast.error('That snippet is private or does not exist.')
      } else {
        toast.error('Failed to load shared snippet.')
      }
    })
}, [])  // run once on mount
```

7. Landon - Detailed Task Breakdown

This section is Landon's reference. Backend hardening, schema migrations, controllers, and the rem pseudo-instruction. Each subsection has the goal, the acceptance criteria, and the code.

7.1 Secret rotation and git history scrub

Step-by-step

32. Decide on the new Postgres password offline. Pick something strong (at least 20 chars, mixed case, symbols).
33. In pgAdmin (or psql -U postgres): ALTER USER postgres WITH PASSWORD 'newPasswordHere';
34. Verify: try the new password by reconnecting. Old password should fail.
35. On your local machine, create webmars-api/.env (do NOT commit). Contents: DB_PASSWORD=newPasswordHere on one line, JWT_SECRET=<run openssl rand -base64 48> on another.
36. Add .env to .gitignore if not already present.
37. Create .env.example (DO commit): DB_PASSWORD=changeme\nJWT_SECRET=changeme on separate lines. Document required vars.
38. Rewrite application.properties to use \${VAR:default} syntax (snippet in Section 3.5 of this doc).
39. Boot the app locally with the env vars set. Confirm it connects and that JWT_SECRET injection works.

Scrubbing git history

DESTRUCTIVE
The following rewrites history and will conflict with any clone of the repository that has not been re-cloned afterward. Coordinate with the team in Discord/Slack before running it.

```
# Install git-filter-repo (more modern than filter-branch).
# macOS:  brew install git-filter-repo
# Linux:  pip install git-filter-repo

# Create a replacements file listing every secret to scrub.
cat > /tmp/replacements.txt <<'EOF'
JHP715BaLL$M@ui==>REMOVED_DB_PASSWORD
webmars-super-secret-key-must-be-32-bytes!==>REMOVED_JWT_SECRET
EOF

# Run the scrub.
git filter-repo --replace-text /tmp/replacements.txt

# Force-push to GitHub.
git push --force --all
git push --force --tags
```

```
# Tell every team member with a clone:
# Delete your local clone and re-clone from scratch. Do not git pull;
# the histories are now incompatible.
```

7.2 CORS configuration

Goal: the deployed frontend can call the deployed backend. The dev frontend (localhost:5173) can call the dev backend (localhost:8080).

Full SecurityConfig.java

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    private final JwtFilter jwtFilter;

    public SecurityConfig(JwtFilter jwtFilter) {
        this.jwtFilter = jwtFilter;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .cors(Customizer.withDefaults())
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/snippets/*").permitAll()
                .requestMatchers(HttpMethod.GET, "/snippets/public").permitAll()
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration cfg = new CorsConfiguration();
        cfg.setAllowedOrigins(List.of(
            "http://localhost:5173",
            "http://localhost:5174",
            "https://www.webmarsimulator.com",
            "https://webmarsimulator.com"
        ));
        cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        cfg.setAllowedHeaders(List.of("Authorization", "Content-Type"));
        cfg.setExposedHeaders(List.of("Authorization"));
        cfg.setAllowCredentials(true);
        cfg.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource src = new UrlBasedCorsConfigurationSource();
        src.registerCorsConfiguration("/**", cfg);
        return src;
    }

    @Bean
```

```
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

7.3 The Snippet entity for v1.2

```
@Entity
@Table(name = "snippets")
public class Snippet {

    public enum Visibility { PUBLIC, PRIVATE }

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    @Column(nullable = false, columnDefinition = "TEXT")
    private String code;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "owner_id")
    @JsonIgnoreProperties({"password", "createdAt"})
    private User owner;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 10)
    private Visibility visibility = Visibility.PRIVATE;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at", nullable = false)
    private LocalDateTime updatedAt;

    @PrePersist
    void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = createdAt;
    }

    @PreUpdate
    void onUpdate() {
        updatedAt = LocalDateTime.now();
    }

    // getters and setters omitted for brevity
}
```

7.4 The Run entity and RunRepository projection

```
@Entity
@Table(name = "runs")
public class Run {
    public enum ExitStatus { COMPLETED, ERROR, PAUSED, ABORTED }

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "snippet_id", nullable = false)
    private Snippet snippet;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(name = "started_at", nullable = false, updatable = false)
    private LocalDateTime startedAt;

    @Column(name = "duration_ms")
    private Integer durationMs;

    @Column(name = "instructions_executed")
    private Integer instructionsExecuted;

    @Enumerated(EnumType.STRING)
    @Column(name = "exit_status", length = 20)
    private ExitStatus exitStatus;

    @Column(name = "error_message", columnDefinition = "TEXT")
    private String errorMessage;

    @PrePersist
    void onCreate() { startedAt = LocalDateTime.now(); }

    // getters and setters omitted
}
```

The most-run projection query

```
public interface RunRepository extends JpaRepository<Run, Long> {

    List<Run> findByUserIdOrderByStartedAtDesc(Long userId, Pageable pageable);

    @Query("""
        SELECT r.snippet.id   AS snippetId,
               r.snippet.title AS title,
               COUNT(r)        AS runCount,
               MAX(r.startedAt) AS lastRun
        FROM Run r
        WHERE r.user.id = :userId
        GROUP BY r.snippet.id, r.snippet.title
        ORDER BY COUNT(r) DESC
    """)
    List<MostRunProjection> findMostRunByUser(@Param("userId") Long userId, Pageable
pageable);
```

```
    interface MostRunProjection {
        Long getSnippetId();
        String getTitle();
        Long getRunCount();
        LocalDateTime getLastRun();
    }
}
```

7.5 The rewritten SnippetController

Goal: ownership-aware, visibility-gated, with proper status codes everywhere.

```
@RestController
@RequestMapping("/snippets")
public class SnippetController {

    private final SnippetRepository snippets;
    private final UserRepository users;

    public SnippetController(SnippetRepository snippets, UserRepository users) {
        this.snippets = snippets;
        this.users = users;
    }

    @GetMapping("/{id}")
    public SnippetResponse get(@PathVariable Long id,
                               @AuthenticationPrincipal String username) {
        Snippet s = snippets.findById(id)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
        if (s.getVisibility() == Snippet.Visibility.PRIVATE) {
            if (username == null || !s.getOwner().getUsername().equals(username)) {
                throw new ResponseStatusException(HttpStatus.NOT_FOUND);
            }
        }
        return SnippetResponse.from(s);
    }

    @GetMapping("/mine")
    public List<SnippetResponse> mine(@AuthenticationPrincipal String username) {
        User me = users.findByUsername(username)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED));
        return snippets.findByOwnerIdOrderByUpdatedAtDesc(me.getId())
            .stream().map(SnippetResponse::from).toList();
    }

    @GetMapping("/public")
    public Page<SnippetResponse> publicFeed(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size) {
        return snippets.findByVisibility(
                Snippet.Visibility.PUBLIC,
                PageRequest.of(page, Math.min(size, 100),
                    Sort.by(Sort.Direction.DESC, "updatedAt")))
            .map(SnippetResponse::from);
    }
```

```
    @PostMapping("/save")
    public SnippetResponse save(@Valid @RequestBody SaveSnippetRequest req,
                                @AuthenticationPrincipal String username) {
        User owner = users.findByUsername(username)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED));
        Snippet s = new Snippet();
        s.setTitle(req.title());
        s.setCode(req.code());
        s.setOwner(owner);
        s.setVisibility(req.visibility() == null
            ? Snippet.Visibility.PRIVATE
            : req.visibility());
        return SnippetResponse.from(snippets.save(s));
    }

    @PutMapping("/{id}")
    public SnippetResponse update(@PathVariable Long id,
                                  @Valid @RequestBody UpdateSnippetRequest req,
                                  @AuthenticationPrincipal String username) {
        Snippet s = ownedOr404(id, username);
        if (req.title() != null)      s.setTitle(req.title());
        if (req.code() != null)       s.setCode(req.code());
        if (req.visibility() != null) s.setVisibility(req.visibility());
        return SnippetResponse.from(snippets.save(s));
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id, @AuthenticationPrincipal String username) {
        snippets.delete(ownedOr404(id, username));
    }

    private Snippet ownedOr404(Long id, String username) {
        Snippet s = snippets.findById(id)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
        if (username == null || !s.getOwner().getUsername().equals(username)) {
            throw new ResponseStatusException(HttpStatus.NOT_FOUND);
        }
        return s;
    }
}
```

7.6 The new RunController

```
@RestController
@RequestMapping("/runs")
public class RunController {

    private final RunRepository runs;
    private final SnippetRepository snippets;
    private final UserRepository users;

    public RunController(RunRepository runs, SnippetRepository snippets, UserRepository
users) {
        this.runs = runs;
        this.snippets = snippets;
```

```
        this.users = users;
    }

    @PostMapping
    public RunResponse log(@Valid @RequestBody LogRunRequest req,
                           @AuthenticationPrincipal String username) {
        User me = users.findByUsername(username)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED));
        Snippet s = snippets.findById(req.snippetId())
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
        Run r = new Run();
        r.setUser(me);
        r.setSnippet(s);
        r.setDurationMs(req.durationMs());
        r.setInstructionsExecuted(req.instructionsExecuted());
        r.setExitStatus(req.exitStatus());
        r.setErrorMessage(req.errorMessage());
        return RunResponse.from(runs.save(r));
    }

    @GetMapping("/recent")
    public List<RunResponse> recent(@AuthenticationPrincipal String username,
                                    @RequestParam(defaultValue = "20") int limit) {
        User me = users.findByUsername(username).orElseThrow();
        return runs.findByUserIdOrderByStartedAtDesc(
                me.getId(), PageRequest.of(0, Math.min(limit, 100)))
            .stream().map(RunResponse::from).toList();
    }

    @GetMapping("/most-run")
    public List<MostRunResponse> mostRun(@AuthenticationPrincipal String username,
                                         @RequestParam(defaultValue = "10") int limit) {
        User me = users.findByUsername(username).orElseThrow();
        return runs.findMostRunByUser(me.getId(), PageRequest.of(0, Math.min(limit, 100)))
            .stream().map(MostRunResponse::from).toList();
    }
}
```

7.7 The rem pseudo-instruction

Goal: rem $rd, $rs, $rt computes $rd = $rs mod $rt. Expands to div $rs, $rt followed by mfhi $rd. Mirror the existing mul pseudo-instruction which already follows this pattern.

File 1 of 3: src/core/assembler/lexer.ts (line ~63)

```
// In the MNEMONICS Set:
// Pseudo
"li", "la", "move", "nop", "syscall", "break",
"blt", "ble", "bgt", "bge",
"mul", "neg", "not",
"rem",            // <-- ADD
"push", "pop",
```

File 2 of 3: src/core/assembler/parser.ts (line ~108)

```
// In OPERAND_SIGNATURES:
// Pseudo
li:    ["R","I"],
la:    ["R","L"],
move:  ["R","R"],
mul:   ["R","R","R"],
rem:   ["R","R","R"],     // <-- ADD
neg:   ["R","R"], not: ["R","R"],
```

File 3 of 3: src/core/assembler/assembler.ts (after the mul case, around line 374)

```
// ── Pseudo: rem rd, rs, rt  →  div rs, rt  +  mfhi rd ──────────────────────
case "rem": return [
  "000000" + reg(op1) + reg(op2) + "0000000000011010",  // div rs, rt  (funct=011010)
  "0000000000000000" + reg(op0) + "00000010000",         // mfhi rd     (funct=010000)
];
```

8. Zachary - Detailed Task Breakdown

This section is Zachary's reference. Tool fixes, the run-logging hook that bridges the simulator and the backend, and the bulk of the test coverage.

8.1 BitmapDisplay 8x8 (and 16x16)

Goal: add the two missing grid sizes that students need for hand-crafting small pixel-art demos. Trivial change, one PR.

The one-line change

File: src/ui/tools/BitmapDisplay.tsx, line 16.

```
// BEFORE
const DIMENSION_OPTIONS = [32, 64, 128, 256] as const

// AFTER
const DIMENSION_OPTIONS = [8, 16, 32, 64, 128, 256] as const
```

Smoke test before pushing

40. npm run dev
41. Tools menu -> Bitmap Display
42. Pick 8x8 grid + 32px cell size. Confirm canvas is 256x256 px.
43. Add a .data block to a test program with 64 .word entries with different RGB values. Run it. Confirm the 64 colored pixels render in row-major order.
44. Pick 16x16, confirm 256 pixels render correctly.
45. Verify the existing 32/64/128/256 still work.

8.2 Screen magnifier rewrite (DOM-clone approach)

Goal: replace the current placeholder with a working magnifier. The chosen approach is to clone the DOM element under the cursor into the loupe container and let the browser render it twice via CSS transform. No html2canvas dependency, no Monaco compatibility issues.

Acceptance criteria

- Toggling the magnifier on shows a 240x160 px loupe near the cursor.
- Moving the cursor over any element marked data-magnify-region shows that element magnified 2x.
- Hovering over an element NOT marked data-magnify-region (or over empty space) shows a hint: 'Hover a panel to magnify'.
- Escape closes the magnifier.
- The magnifier never traps the mouse - pointerEvents: none on the loupe.
- Doesn't cause CPU spikes - re-clones only when target element changes, not on every mousemove.

Tag the panels to magnify (Day 2 prep)

Add data-magnify-region to the OUTER wrapper of each panel users would want to magnify:

- src/ui/RegisterTable.tsx - the table wrapper div
- src/ui/MemoryPanel.tsx - the main panel container
- src/ui/ConsolePanel.tsx - the console output area
- src/ui/StatusBar.tsx - the bar element
- src/ui/FpuRegisterTable.tsx - the table wrapper
- src/ui/MessagesPanel.tsx, src/ui/SymbolsPanel.tsx, src/ui/BreakpointsPanel.tsx - whichever have small text users would zoom into

Skip the Monaco editor on purpose - its DOM is too dynamic for a stable clone.

The rewritten component

```
import { useEffect, useRef, useState } from 'react'
import { useSimulator } from '@/hooks/useSimulator.ts'

const ZOOM = 2
const SIZE_W = 240
const SIZE_H = 160

export function ScreenMagnifier() {
  const on     = useSimulator((s) => s.screenMagnifierOn)
  const toggle = useSimulator((s) => s.toggleScreenMagnifier)
  const [pos, setPos] = useState({ x: 0, y: 0 })
  const containerRef = useRef<HTMLDivElement>(null)
  const lastTargetRef = useRef<Element | null>(null)

  // Mouse and Escape wiring.
  useEffect(() => {
    if (!on) return
    const onMove = (e: MouseEvent) => setPos({ x: e.clientX, y: e.clientY })
    const onKey  = (e: KeyboardEvent) => { if (e.key === 'Escape') toggle() }
    window.addEventListener('mousemove', onMove)
    window.addEventListener('keydown', onKey)
    return () => {
      window.removeEventListener('mousemove', onMove)
      window.removeEventListener('keydown', onKey)
    }
  }, [on, toggle])

  // Pick the element under the cursor and clone it into the loupe.
  // Re-clone only when the target changes - we use the closest()
  // walk to a region root so small movements within the same panel
  // do not retrigger a clone.
  useEffect(() => {
    if (!on || !containerRef.current) return
    const under = document.elementFromPoint(pos.x, pos.y)
    const target = under?.closest('[data-magnify-region]') as Element | null
    if (target !== lastTargetRef.current) {
      containerRef.current.innerHTML = ''
      if (target) {
        const cloned = target.cloneNode(true) as HTMLElement
        // Strip interactive bits from the clone so they don't fire
        // accidentally - we only want a visual snapshot.
        cloned.querySelectorAll('button, input, select, textarea')
```

```
              .forEach(el => (el as HTMLElement).style.pointerEvents = 'none')
        containerRef.current.appendChild(cloned)
      }
      lastTargetRef.current = target
    }
  }, [on, pos.x, pos.y])

  if (!on) return null

  // Position the loupe just below and right of the cursor.
  const clientX = Math.min(window.innerWidth  - SIZE_W - 12, pos.x + 24)
  const clientY = Math.min(window.innerHeight - SIZE_H - 12, pos.y + 24)

  const hasTarget = lastTargetRef.current !== null
  const rect = lastTargetRef.current?.getBoundingClientRect()

  // Compute translate so the cursor's position in the source element
  // ends up centered in the loupe at ZOOM scale.
  const tx = rect ? -(pos.x - rect.left) + SIZE_W / (2 * ZOOM) : 0
  const ty = rect ? -(pos.y - rect.top)  + SIZE_H / (2 * ZOOM) : 0

  return (
    <div
      aria-hidden="true"
      style={{
        position: 'fixed', left: clientX, top: clientY,
        width: SIZE_W, height: SIZE_H,
        pointerEvents: 'none', zIndex: 70,
        border: '2px solid var(--accent)', borderRadius: '8px',
        boxShadow: '0 12px 40px rgba(0,0,0,0.45)',
        overflow: 'hidden', background: 'var(--surface-elev)',
      }}
    >
      {hasTarget ? (
        <div
          ref={containerRef}
          style={{
            transform: `scale(${ZOOM}) translate(${tx}px, ${ty}px)`,
            transformOrigin: '0 0',
            pointerEvents: 'none',
          }}
        />
      ) : (
        <div className="flex h-full w-full items-center justify-center
                        text-[11px] text-ink-3 px-3 text-center">
          Hover a panel to magnify
        </div>
      )}
    </div>
  )
}
```

KNOWN LIMITATION

Monaco editor is intentionally NOT marked data-magnify-region. Its DOM is virtual (only visible lines exist in the DOM) and the contenteditable surface has internal state that doesn't survive a clone. Users hovering the editor see the 'Hover a panel to magnify' hint - direct them to OS-level zoom (Cmd/Ctrl + Plus) in the help dialog.

8.3 Run logging hook

Goal: every time a program finishes running (completion, error, abort), log a Run record to the backend. Only for authenticated users who have saved the snippet (otherwise there's no snippetId).

Where to hook in

Find the simulator's run lifecycle in src/hooks/useSimulator.ts. Look for the action that's called when a program halts - likely something like setStatus('halted') or a finalizeRun helper. Add a call to logRunIfAuthenticated() at that point.

Implementation sketch

```
// In useSimulator.ts, add a helper:
async function logRunIfPossible(get: () => SimulatorStore, exitStatus: ExitStatus,
errorMessage?: string) {
  const s = get()
  const snippetId = s.currentSnippetId
  const isAuthed = !!s.authToken
  if (!snippetId || !isAuthed) return  // fire-and-forget; skip if not eligible

  // Compute the duration from the start time we recorded when run() began.
  const durationMs = s.runStartedAt
    ? Date.now() - s.runStartedAt
    : undefined

  try {
    await api.logRun({
      snippetId,
      durationMs,
      instructionsExecuted: s.instructionsExecuted,
      exitStatus,
      errorMessage,
    })
  } catch (err) {
    // Never block the simulator on the log call. Just console.warn.
    console.warn('Failed to log run:', err)
  }
}

// At each place where a run ends:
// On clean completion:
logRunIfPossible(get, 'COMPLETED')
// On error:
logRunIfPossible(get, 'ERROR', errorMessage)
// On user-initiated abort:
logRunIfPossible(get, 'ABORTED')
// On pause (only if we want pause-as-end, otherwise skip):
// logRunIfPossible(get, 'PAUSED')
```

Add runStartedAt: number | null to the store, set in the run() action's first line, used by logRunIfPossible to compute duration.

8.4 Integration tests with Testcontainers

Goal: the test suite spins up a real ephemeral PostgreSQL instance, runs Flyway, exercises every controller endpoint, and tears down. CI-friendly, no external dependencies.

pom.xml additions

```
<!-- inside <dependencies> -->
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers</artifactId>
    <version>1.20.4</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <version>1.20.4</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>1.20.4</version>
    <scope>test</scope>
</dependency>
```

Shared test base

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@Testcontainers
@AutoConfigureMockMvc
public abstract class PostgresTestBase {

    @Container
    @SuppressWarnings("resource")
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:18")
        .withDatabaseName("webmars_test")
        .withUsername("test")
        .withPassword("test");

    @DynamicPropertySource
    static void registerProperties(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url",      postgres::getJdbcUrl);
        r.add("spring.datasource.username", postgres::getUsername);
        r.add("spring.datasource.password", postgres::getPassword);
        r.add("spring.jpa.hibernate.ddl-auto", () -> "validate");
        r.add("JWT_SECRET", () -> "test-secret-must-be-at-least-32-bytes-long-for-hs256");
    }

    @Autowired protected MockMvc mvc;
    @Autowired protected ObjectMapper json;
    @Autowired protected UserRepository users;
    @Autowired protected SnippetRepository snippets;
    @Autowired protected RunRepository runs;

    @AfterEach
    void cleanDb() {
        runs.deleteAll();
        snippets.deleteAll();
```

```
        users.deleteAll();
    }
}
```

Example test - SnippetControllerTest

```
class SnippetControllerTest extends PostgresTestBase {

    @Test
    void privateSnippet_returns404_toNonOwner() throws Exception {
        // Setup: alice registers, saves a private snippet.
        String aliceToken = registerAndLogin("alice", "password123");
        Long snippetId = saveSnippet(aliceToken, "alice's private", "li $t0, 1",
"PRIVATE");

        // Bob registers and logs in.
        String bobToken = registerAndLogin("bob", "password123");

        // Bob tries to fetch alice's private snippet.
        mvc.perform(get("/snippets/" + snippetId)
                .header("Authorization", "Bearer " + bobToken))
            .andExpect(status().isNotFound());
    }

    @Test
    void publicSnippet_isReadable_byAnonymous() throws Exception {
        String aliceToken = registerAndLogin("alice", "password123");
        Long id = saveSnippet(aliceToken, "public hello", "li $t0, 42", "PUBLIC");

        mvc.perform(get("/snippets/" + id))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.code").value("li $t0, 42"));
    }

    @Test
    void delete_byNonOwner_returns404() throws Exception {
        String aliceToken = registerAndLogin("alice", "password123");
        Long id = saveSnippet(aliceToken, "title", "code", "PUBLIC");
        String bobToken = registerAndLogin("bob", "password123");

        mvc.perform(delete("/snippets/" + id)
                .header("Authorization", "Bearer " + bobToken))
            .andExpect(status().isNotFound());

        // Confirm not actually deleted.
        assertThat(snippets.findById(id)).isPresent();
    }

    // helpers omitted: registerAndLogin, saveSnippet
}
```

8.5 Frontend tests

Vitest is already configured. Add three areas:

- src/tests/api.test.ts - mock fetch, assert auth header is set on requests, ApiError is thrown on non-2xx.
- src/tests/assembler.test.ts - the three rem tests (expansion, positive remainder, negative dividend).
- src/tests/persist.test.ts - verify that setting source updates localStorage, and that hydrating the store from a payload restores source.

9. Testing and QA Strategy

Three layers of testing, ordered cheapest to most expensive: backend integration tests with Testcontainers, frontend Vitest unit tests, and manual QA against the running app. Each layer catches bugs the others miss.

9.1 Backend integration tests (Zachary, Days 7-8)

Stack: Spring Boot Test, MockMvc, Testcontainers, JUnit 5. Tests spin up an ephemeral PostgreSQL container, run Flyway migrations against it, exercise the controllers via MockMvc, and tear down. No external dependencies, runs cleanly in CI.

Required test classes

Class	Scenarios
AuthControllerTest	Register success; register with duplicate username returns 409; register with invalid username returns 400; login success; wrong password returns 401; unknown user returns 401.
SnippetControllerTest	Owner can read private; non-owner gets 404 on private; anonymous can read public; non-owner gets 404 on delete; owner can update; PUT preserves owner; visibility toggle works.
RunControllerTest	Logging a run sets user from JWT not from body; recent and most-run scoped to authenticated user; non-existent snippet returns 404.
JwtUtilTest	Round-trip a token; reject tampered token; reject expired token; reject token signed with a different secret; throw on too-short secret at startup.

9.2 Frontend unit tests (Zachary, Day 9)

- src/tests/api.test.ts - mock fetch, assert auth header is attached when a token exists, assert ApiError is thrown with the right status on non-2xx responses.
- src/tests/assembler.test.ts - the three rem cases (expansion, positive remainder, negative dividend). Use the existing golden-program harness.
- src/tests/persist.test.ts - simulate a store creation, mutate source, read localStorage manually and assert the key webmars-store-v1 was written. Then create a fresh store and assert source rehydrates.

9.3 Manual QA checklist (Zachary, run twice: Day 9 against dev, Day 10 against production)

Each item below is a step that must succeed. If any step fails, file a GitHub issue with screenshots before continuing.

Authentication

46. Open the site in a private/incognito window. Click 'Sign in'. Switch to Register tab. Create username 'qatest1' password 'password123'. Confirm a welcome toast appears and the top-right shows the username.
47. Refresh the page. Confirm you are still logged in (token persisted).
48. Click the username -> Log out. Confirm the Sign in button reappears.
49. Click Sign in -> Login tab. Enter qatest1 / wrongpassword. Confirm a toast: 'Wrong username or password.' Status code in DevTools network tab is 401, not 500.
50. Enter correct credentials. Confirm successful login.
51. Try to register with username 'qa' (too short). Confirm a validation error appears with a clear message.

Persistence (the bug the user reported)

52. Open the editor. Type 'addi $t0, $zero, 999' into the source.
53. Press F5 to refresh.
54. Confirm the source is preserved. This is the fix for Bug 2.1.
55. Press the Reset button to clear simulator state. Confirm registers go to zero. Refresh again. Confirm code is still preserved AND registers are zero on load (simulator state NOT persisted).

Light mode (the other bug the user reported)

56. Open SettingsDialog. Pick the Light theme option.
57. Confirm the entire UI - including the Monaco editor - switches to a light color scheme. This is the fix for Bug 2.2.
58. Switch back to Dark. Confirm the editor goes dark.
59. Switch to System. Toggle OS-level dark mode in macOS / Windows. Confirm the editor follows.

Save and share

60. Logged in as qatest1. Type a unique snippet ('addi $t0, $zero, 42'). Click Save.
61. In the Save dialog, set title to 'qa-test-public' and visibility to PUBLIC. Click Save.
62. Confirm the Share modal appears with a copyable link like https://webmarsimulator.com/?snippet=N.
63. Click Copy. Confirm a toast 'Copied to clipboard'.
64. Open the copied URL in a fresh incognito window (or a different browser). Confirm the snippet loads and you can read the code.
65. As qatest1 in the original window, edit the snippet code and click Save again. Confirm no Share modal appears (snippet already saved), and the change persists. Refresh the incognito window - confirm it sees the update.

Visibility gating

66. As qatest1, save a new snippet with visibility=PRIVATE. Copy the share link.
67. Open the link in an incognito window. Confirm you get a 'snippet is private or does not exist' message, NOT the snippet content.

68. In the incognito window, log in as a different user qatest2. Try to load the same private snippet URL. Confirm you still get 404 (qatest2 is not the owner).

Run history

69. As qatest1, load a saved snippet. Press F5 (Run).
70. Open the History panel in the right inspector. Confirm 'Recent Runs' shows the run you just did with timestamp and status.
71. Run the same snippet 5 more times.
72. Confirm Most Run shows this snippet at the top with runCount: 6.

Delete ownership (the security bug)

73. As qatest2, navigate to qatest1's public snippet. Confirm you can view it.
74. Open DevTools, copy the snippet id from the URL. Send a DELETE /snippets/{id} request manually with qatest2's token.
75. Confirm response is 404 (not 200, not 403). qatest1's snippet must still be intact - verify by reloading the page.

Tools fixes

76. Tools -> Bitmap Display. Select 8x8 grid. Confirm the canvas renders 64 pixels.
77. Select 16x16. Confirm 256 pixels.
78. Tools -> Screen Magnifier. Toggle on. Hover over the register table. Confirm registers appear magnified 2x. Hover over the editor - confirm 'Hover a panel to magnify' hint appears (Monaco intentionally not magnified). Press Escape, magnifier closes.
79. Type 'rem $t0, $t1, $t2' into the editor. Assemble (F3). Confirm no errors.
80. Load values into $t1 and $t2 via li, run the program, confirm $t0 contains the correct remainder.

Error handling

81. Stop the backend (kill the local server, or in production: open Render dashboard and pause the service). Try to save a snippet. Confirm an error toast appears, NOT a silent failure.
82. Click Save twice rapidly. Confirm the button disables after the first click and a duplicate request is not sent (loading state working).

DEFINITION OF DONE

A PR is mergeable when: code review is approved by a different engineer; all existing tests still pass; any new behavior has a test (where applicable); the relevant section of this manual QA checklist passes for the changed area. PRs that fail any of these get sent back for changes - even if the change is trivially correct in isolation.

10. Deployment Plan and Risk Register

This section covers the production deployment, the risks the team has identified going into the sprint, and the contingency for each.

10.1 Pre-launch checklist

Run this exact list before pressing 'Deploy' on either repo. Every box should be checked off; no exceptions.

83. BACKEND: hardcoded secrets removed from current code AND git history (filter-repo run, force-pushed, team re-cloned).
84. BACKEND: JWT_SECRET is freshly generated (openssl rand -base64 48), at least 32 bytes, set as a Render environment variable, NOT visible in any file.
85. BACKEND: DB_PASSWORD set as a Render environment variable.
86. BACKEND: spring.jpa.show-sql is false in production (no SQL leaking to logs).
87. BACKEND: spring.jpa.hibernate.ddl-auto is 'validate' not 'update' (Flyway runs the schema, JPA only validates).
88. BACKEND: CORS allowed-origins includes the production Vercel domain (https://webmarsimulator.com and https://www.webmarsimulator.com) and ONLY those - no wildcards.
89. BACKEND: all integration tests pass on main branch (./mvnw verify).
90. BACKEND: rate limiter active on /auth/** (5 attempts per username per 15 min).
91. FRONTEND: VITE_API_BASE environment variable set in Vercel project settings, pointing at the Render service URL.
92. FRONTEND: npm run lint clean, npm run typecheck clean, npm test clean on main.
93. FRONTEND: production build succeeds (npm run build) without warnings.
94. FRONTEND: bundle size review - confirm no accidental large additions (no html2canvas, no full lodash, etc.).
95. BOTH: Bryan and Landon are available for the deploy window in case something needs hotfixing immediately.

10.2 Deploy day runbook (Day 10, Friday afternoon)

T-2 hours: prep

- Landon: merge any outstanding PRs to backend main. Tag the release: git tag v1.2.0 && git push --tags.
- Bryan: merge any outstanding PRs to frontend main. Tag the release similarly.
- Zachary: one final pass of the manual QA checklist (Section 9.3) against the dev environment.

T-1 hour: backend deploy

96. Landon: from Render dashboard, click 'Manual Deploy' or push to main if auto-deploy is configured.

97. Watch the build log. First Maven build can take 8-12 minutes.
98. Once 'Live', curl https://webmars-api.onrender.com/snippets/1 from a terminal. Expected: HTTP 404 (snippet doesn't exist), NOT a connection error.
99. If 404 received: backend is live. Notify team in Discord.

T-0: frontend deploy

100. Bryan: in Vercel dashboard, trigger a production deploy (or it auto-deploys on push to main).
101. Wait 2-3 minutes for the build.
102. Visit https://webmarsimulator.com in a fresh incognito window. Page should load.
103. Open DevTools network tab. Click Sign in. Verify the POST hits webmars-api.onrender.com (not localhost), and returns 200.

T+30 min: smoke test

- Zachary runs the production-facing checklist from Section 9.3, items 1-15 (auth, save, share, run history).
- Any failure: file a hotfix GitHub Issue, triage with the team, fix Saturday or Sunday morning if non-critical.

T+1 hour: announce

- Post in course Slack/Discord/Canvas thread: 'WebMARS v1.2 is live at webmarsimulator.com.'
- Include the demo gif/video Zachary recorded.

10.3 Risk register

Risk	Likelihood	Impact	Mitigation
Render free tier cold starts (30s+) annoy users	High	Medium	Acceptable for a course project. Document the tradeoff in the README. Upgrade to paid tier if usage demands.
Render free Postgres expires after 90 days	Certain	High	Set a calendar reminder. Migrate to a paid plan or different host before the expiry.
JWT secret accidentally committed during the sprint	Low	Critical	Pre-commit hook that greps for known secret patterns. Rotate immediately if found.
Flyway migration fails on production DB	Medium	High	Test migrations against a fresh local Postgres before deploying. Have a rollback SQL script ready.

Risk	Likelihood	Impact	Mitigation
CORS misconfiguration blocks the frontend	Medium	High	Test the deployed combo in a fresh incognito window before announcing. Have Landon on-call during the deploy.
Monaco light theme has unreadable token colors	Medium	Low	Hand-test the theme on 2-3 example MIPS programs before merging. Iterate on token colors if anything is unreadable.
Magnifier rewrite breaks for some panels	Medium	Low	It's an opt-in tool. Worst case: revert and ship with the placeholder. Document known limitations in the help dialog.
A team member is unavailable mid-sprint	Medium	High	Every PR has at least one reviewer who could pick up the next item. No work-in-progress sits on a local machine for more than 24 hours.
Backend deploy fails 5 minutes before deadline	Low	High	Deploy on Day 10 morning, not afternoon. Buffer the entire weekend for hotfixes.

10.4 Backlog for v1.3

Items that surfaced during planning but did not fit the 2-week sprint. These should become GitHub issues tagged 'v1.3' after the v1.2 deploy.

- Refresh tokens - access tokens expire in 8 hours; add long-lived HttpOnly refresh tokens so logged-in users stay logged in across days.
- Email field on User + password reset flow. Currently if you forget your password, you lose the account. Add forgot-password via email.
- Snippet versioning - keep a history of edits to a snippet (like Git for MIPS). Useful for showing students how their solution evolved.
- Comments on public snippets - lightweight discussion thread per snippet.
- Forking - 'Save a copy to my account' button on someone else's public snippet.
- Examples curation - star or mark snippets as 'editor's pick' for the public feed landing page.
- Profile pages - /u/<username> showing that user's public snippets.
- API rate limit for /snippets/** endpoints (not just /auth/**).
- OAuth login via GitHub - convenient for the target audience (CS students).
- Magnifier: support the Monaco editor via a different strategy (probably a hidden Monaco instance kept in sync via the model API).

10.5 Appendix - file change summary

Backend files touched in this sprint

```
webmars-api/src/main/java/com/webmars/webmars_api/
├── SecurityConfig.java                  MODIFY - CORS, public GET, validation
├── JwtUtil.java                         MODIFY - env var secret, 8h expiry
├── AuthController.java                  MODIFY - DTOs, proper status codes
├── User.java                            MODIFY - @JsonIgnore on password
├── Snippet.java                         MODIFY - owner, visibility, updated_at
├── SnippetRepository.java               MODIFY - findByOwnerId, findByVisibility
├── SnippetController.java               REWRITE - full v1.2 endpoints
├── Run.java                             NEW
├── RunRepository.java                   NEW
├── RunController.java                   NEW
├── GlobalExceptionHandler.java          NEW
├── RateLimitFilter.java                 NEW
└── dto/
    ├── LoginRequest.java                NEW
    ├── RegisterRequest.java             NEW
    ├── UserResponse.java                NEW
    ├── SaveSnippetRequest.java          NEW
    ├── UpdateSnippetRequest.java        NEW
    ├── SnippetResponse.java             NEW
    ├── LogRunRequest.java               NEW
    ├── RunResponse.java                 NEW
    └── MostRunResponse.java             NEW

webmars-api/src/main/resources/
├── application.properties               MODIFY - env-var indirection
└── db/migration/
    ├── V1__init.sql                     NEW
    ├── V2__snippet_ownership.sql        NEW
    └── V3__runs.sql                     NEW

webmars-api/pom.xml                     MODIFY - +validation, +flyway, +bucket4j,
+testcontainers
webmars-api/.env.example                NEW
webmars-api/.gitignore                  MODIFY - add .env
```

Frontend files touched in this sprint

```
WebMARS-main/src/
├── lib/
│   ├── api.ts                           NEW - HTTP client
│   └── router.ts                        MODIFY - parse ?snippet=<id>
├── hooks/
│   ├── useSimulator.ts                  MODIFY - persist middleware, auth slice,
│   │                                    themePreference, run-logging hook
│   └── useSystemColorScheme.ts          NEW
├── ui/
│   ├── AuthModal.tsx                    NEW
│   ├── ShareModal.tsx                   NEW
│   ├── MySnippetsDrawer.tsx             NEW
│   ├── HistoryPanel.tsx                 NEW
│   ├── PublicFeedModal.tsx              NEW
│   ├── ConfirmDialog.tsx                NEW
│   ├── Toolbar.tsx                      MODIFY - Sign in, Share, Sign out
│   ├── HelpDialog.tsx                   MODIFY - rem entry, magnifier docs
```

```
│   ├── SettingsDialog.tsx              MODIFY - theme picker
│   ├── CodeEditor.tsx                  MODIFY - effectiveTheme, webmars-light
│   ├── RegisterTable.tsx               MODIFY - data-magnify-region
│   ├── MemoryPanel.tsx                 MODIFY - data-magnify-region
│   ├── ConsolePanel.tsx                MODIFY - data-magnify-region
│   ├── StatusBar.tsx                   MODIFY - data-magnify-region
│   └── tools/
│       ├── BitmapDisplay.tsx           MODIFY - add 8, 16 to DIMENSION_OPTIONS
│       └── ScreenMagnifier.tsx         REWRITE - DOM-clone implementation
├── core/assembler/
│   ├── lexer.ts                        MODIFY - rem mnemonic
│   ├── parser.ts                       MODIFY - rem signature
│   └── assembler.ts                    MODIFY - rem expansion
├── App.tsx                             MODIFY - URL param load, dark class toggle
└── tests/
    ├── api.test.ts                     NEW
    ├── persist.test.ts                 NEW
    └── assembler.test.ts               MODIFY - +rem tests

WebMARS-main/
├── .env.local.example                  NEW
├── package.json                        MODIFY - +sonner, +@playwright/test (optional)
└── vercel.json                         (no change - env vars set in Vercel UI)
```

10.6 Reading order for the new contributor

104. Both repos' README.md files - get the lay of the land.
105. WebMARS-main/docs/PRD.md - the original v1.0 product spec.
106. This document, end to end.
107. Your assigned section in this doc (6, 7, or 8) re-read carefully.
108. src/hooks/useSimulator.ts - the heart of the frontend.
109. webmars-api/.../SecurityConfig.java - the heart of the backend.
110. Slack/Discord scrollback for the team's working conventions.

CLOSING NOTE

This document is a living plan, not a contract. If you discover something during implementation that makes a section wrong, edit the doc in the same PR as the code change. The version control is what matters - if main has the right code, main has the right plan.